nulllaborg / tt-encoder-motor

Code Issues Pull requests Actions Projects Security and quality Insights

tt-encoder-motor / README.md

NL-leishen create encoder motor 588acb8 · 9 months ago

Preview Code Blame 109 lines (80 loc) · 5.2 KB Raw

TT-Encoder-Motor

Introduction

An encoded motor is a precision-controllable geared DC motor composed of Hall sensors, magnetic disks, and a geared motor. It converts mechanical motion into digital signals through a built-in encoder, enabling precise measurement of parameters such as position, speed, and acceleration.



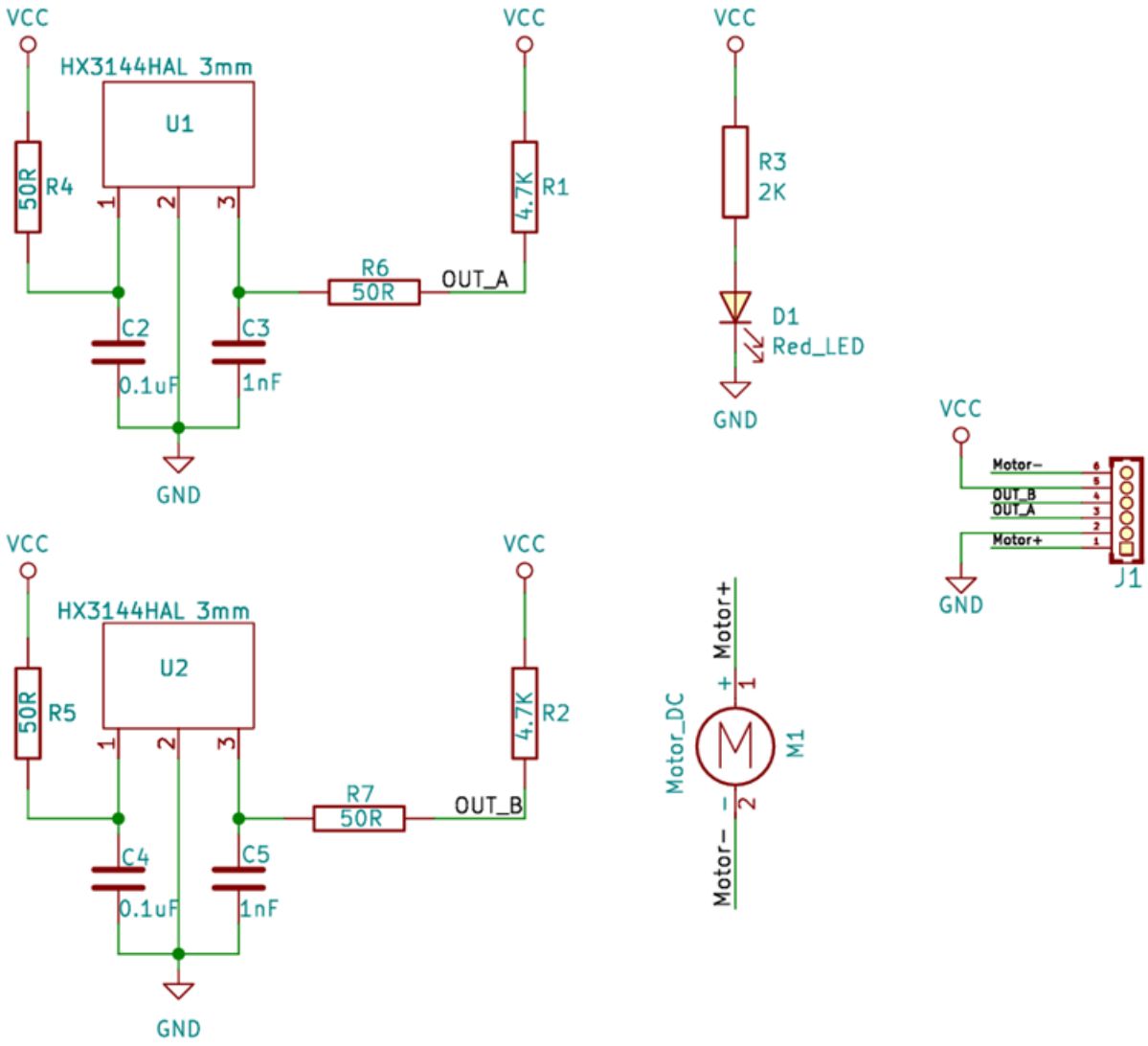


Product Parameter

- Working Voltage: DC 3-12V
- No-load Current: 100mA
- No-load Speed: 113 RPM
- Rated Load Current: 500mA
- Rated Load Speed: 59 RPM
- Rated Load Torque: 1.0 kg·cm
- Stall Current: 0.82A
- Stall Torque: 1.8 kg·cm

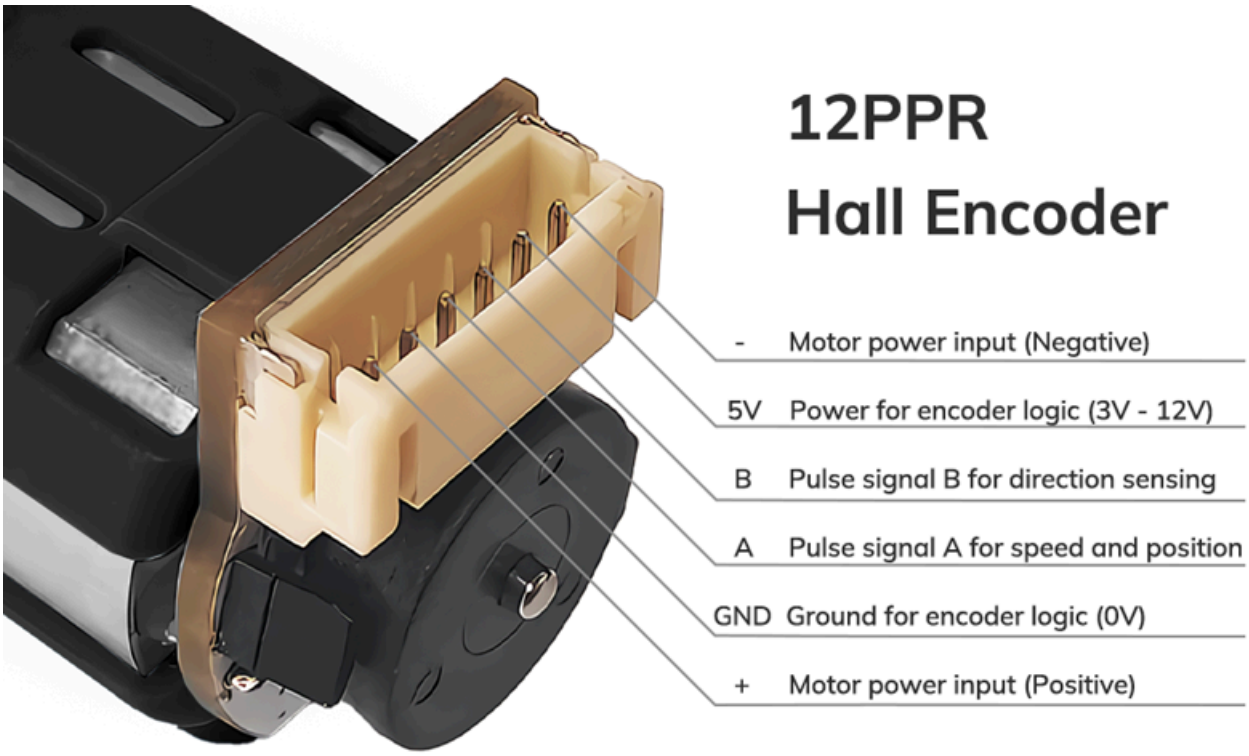
- Interface: PH2.0 Pitch Connector
- Encoder Power Supply: 3.3-5V
- Gear Ratio: 1:90
- Encoder Disk: 12 PPR

Schematic Diagram of Hall Effect Encoder Disc

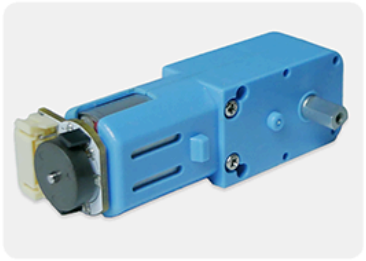
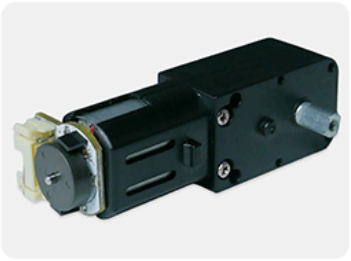


Click here download schematic

Interface Definition



Comparison Diagram

			
Feature	Standard TT Motor	**Half-Metal Encoder TT Motor**	**Full-Metal Encoder TT Motor**
Encoder	/	**12PPR Hall Encoder**	**12PPR Hall Encoder**
Gear Box Material	Plastic	**Metal + Nylon**	**Full Metal**
Output Shaft	Plastic	**Metal**	**Metal**
Control Logic	Open-loop (Voltage only)	**Closed-loop (with Encoder)**	**Closed-loop (with Encoder)**
Torque Capacity	Low (fragile gears)	**1:90 High Torque 12ppr**	**1:90 High Torque 12ppr**
Synchronicity	Poor (wheels drift easily)	**Perfect (Real-time sync)**	**Perfect (Real-time sync)**
Application	Entry-level DIY toys	**Professional Robotics / SLAM**	**Professional Robotics / SLAM**

Parameter Analysis

PPR: Pulses Per Revolution,Number of Raw Pulse Signals Generated Per Revolution of the Encoder Disc

Ratio: Gear Ratio

Output Shaft Resolution: The total number of quadrature counts registered by the encoder interface for one complete (360°) rotation of the motor's final output shaft. This value incorporates the encoder's base PPR, the gearbox reduction ratio, and the quadrature decoding multiplier.

Formula: `CPR = Encoder PPR × Gear Ratio × Quadrature Multiplier`

Unit: Counts per revolution (CPR)

Feature	Single-Frequency (1×)	Double-Frequency (2×)	Quadrature (4×)
English Term	Single-Frequency / 1× Mode	Double-Frequency / 2× Mode	Quadrature (4×) Mode
Counting Principle	Counts only **rising edge of Channel A**	Counts **rising + falling edges of Channel A**	Counts **all edges of Channels A & B**
Direction Detection	Not supported (unidirectional counting)	Supported (requires additional logic)	**Built-in direction detection**
Resolution Gain	Baseline (1×)	2× baseline	**4× baseline**
Interrupt Frequency	Lowest	Moderate	**Highest**
Typical Application	Simple speed measurement	Moderate-precision control	**High-precision position control**

Arduino Example

Get the Encoder value

```
namespace {
const uint8_t kEncoderPinA = 2;  // Encoder A phase signal input pin
const uint8_t kEncoderPinB = 3;  // Encoder B phase signal input pin

volatile int32_t g_encoder_count = 0;

// Interrupt Service Routine - Handles the rising edge of phase A signal. This function is executed whe
void HandleEncoderInterrupt() {
  // Read the level of phase B signal and determine the direction.
  if (digitalRead(kEncoderPinB) == LOW) {
    // Phase B is at a low level, indicating a forward direction. The count is increased.
    ++g_encoder_count;
  } else {
    // Phase B is at a high level, indicating a reverse direction. The count is decreased.
    --g_encoder_count;
  }
}
}  // namespace

void setup() {
  Serial.begin(115200);

  // Configure the encoder input pins and enable the internal pull-up resistors.
  pinMode(kEncoderPinA, INPUT_PULLUP);
  pinMode(kEncoderPinB, INPUT_PULLUP);
  // Set up an interrupt to detect the rising edge of the phase A signal. Single-Frequency Mode
  attachInterrupt(digitalPinToInterrupt(kEncoderPinA), HandleEncoderInterrupt, RISING);
  Serial.println("Start reading the pulse count....");
}

void loop() {
  Serial.print("Pulse Count: ");
  Serial.println(g_encoder_count);
  Serial.print("Encoder rotation angle: ");
  Serial.print(g_encoder_count * 360.0 / (12 * 90 ));  // 90 is encoder motor ratio  12 is encoder motc
  Serial.println(" °");
```

```
  delay(500);
}
```

This example program demonstrates how to read the pulse signals from an incremental encoder and display the pulse count and encoder rotation angle via the serial monitor. The program uses an interrupt to detect changes in the encoder's phase A signal and determines the rotation direction based on the status of the phase B signal, thereby incrementing the count for forward rotation and decrementing it for reverse rotation.

After the program is uploaded, open the Serial Monitor, set the baud rate to 115200, click the "Open Serial" button, and wait for the program to run. Manually rotate the encoder, and the current pulse count as well as the rotation angle will be displayed in the serial monitor. When rotating the encoder clockwise, both the pulse count and rotation angle increase. When rotating the encoder counterclockwise, both the pulse count and rotation angle decrease.